CSCI-4208: Developing Advanced Web Applications

Week 7: Lecture 10

JavaScript - Events

Events: Overview

1. Events: Event Loop
2. Events: Event Types
3. Events: Listeners & Handlers
4. Events: Event object
5. Events: Bubbling & Capturing
6. Events: Delegation
7. Events: Custom Events
8. Events: Mouse
9. Events: Keyboard
10. Events: Touch
11. Events: Document
12. Events: Form & Input
13. Events: Time

1. Events: Event Loop

Browser JavaScript execution flow, as well as in Node.js, is based on an *event loop*.

Event Loop

The *event loop* concept is very simple. There's an endless loop, where the JavaScript engine waits for tasks, executes them and then sleeps, waiting for more tasks.

The general algorithm of the engine:

1. While there are tasks:
    - execute them, starting with the oldest task.
2. Sleep until a task appears, then go to 1.

That's a formalization for what we see when browsing a page. The JavaScript engine does nothing most of the time, it only runs if a script/handler/event activates.

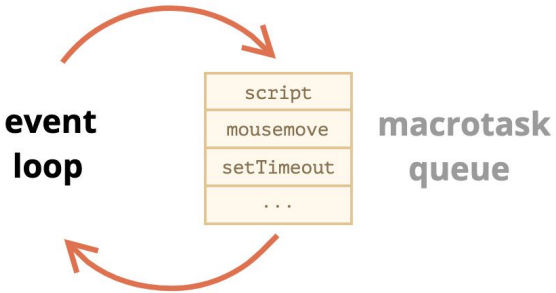
1. Events: Event Loop

Examples of tasks:

- When an external script `<script src="...">` loads, the task is to execute it.
- When a user moves their mouse, the task is to dispatch `mousemove` event and execute handlers.
- When the time is due for a scheduled `setTimeout`, the task is to run its callback.
- ...and so on.

Tasks are set – the engine handles them – then waits for more tasks (while sleeping and consuming close to zero CPU).

It may happen that a task comes while the engine is busy, then it's enqueued.



1. Events: Event Loop

Event Loop is single-threaded

1. Rendering never happens while the engine executes a task. It doesn't matter if the task takes a long time. Changes to the DOM are painted only after the task is complete.

2. If a task takes too long, the browser can't do other tasks, such as processing user events. So after a time, it raises an alert like "Page Unresponsive", suggesting killing the task with the whole page. That happens when there are a lot of complex calculations or a programming error leading to an infinite loop.

2. Events: Event Types

An event is a signal that something has happened. All DOM nodes generate such signals (but events are not limited to DOM).
Here's a list of the most useful DOM events, just to take a look at:

Mouse events:
- `click` – when the mouse clicks on an element (touchscreen devices generate it on a tap).
- `contextmenu` – when the mouse right-clicks on an element.
- `mouseover` / `mouseout` – when the mouse cursor comes over / leaves an element.
- `mousedown` / `mouseup` – when the mouse button is pressed / released over an element.
- `mousemove` – when the mouse is moved.

Keyboard events:
- `keydown` and `keyup` – when a keyboard key is pressed and released.

Form element events:
- `submit` – when the visitor submits a `<form>`.
- `focus` – when the visitor focuses on an element, e.g. on an `<input>`.

Document events:
- `load` – when the HTML is loaded and processed, DOM is fully built.

CSS events:
- `transitionend` – when a CSS-animation finishes.

There are many other events. We'll get into more details of particular events in next chapters.

3. Events: Listeners & Handlers

To react on events we can assign a *handler* – a function that runs in case of an event. Handlers are a way to run JavaScript code in response to user actions.

Event Handlers are callback functions.

> ***Callback function:***
> *A callback function is a function passed into another function as an argument, which is then invoked inside the outer function to complete some action.*

3. Events: Listeners & Handlers

HTML-attribute

A handler can be set in HTML with an attribute named `on<event>`.

For instance, to assign a `click` handler for an `input`, we can use `onclick`, like here:

```
<input value="click me" onclick="console.log('click!')" type="button">
```

On mouse click, the code inside `onclick` runs.

Please note that inside `onclick` we use single quotes, because the attribute itself is in double quotes. If we forget that the code is inside the attribute and use double quotes inside, like this: `onclick="alert("Click!")"`, then it won't work right.

3. Events: Listeners & Handlers

HTML-attribute

An HTML-attribute is not a convenient place to write a lot of code, so we'd better create a JavaScript function and call it there.

Here a click runs the function `countRabbits()`:

```
<script>
  function countRabbits() {
    for(let i=1; i<=3; i++) {
      alert("Rabbit number " + i);
    }
  }
</script>

<input type="button" onclick="countRabbits()" value="Count rabbits!">
```

As we know, HTML attribute names are not case-sensitive, so `ONCLICK` works as well as `onClick` and `onCLICK`… But usually attributes are lowercased: `onclick`.

3. Events: Listeners & Handlers

DOM property

We can assign a handler using a DOM property `on<event>`.

For instance, `elem.onclick`:

```
<input id="btn" value="click me" type="button">
<script>
    btn.onclick = () => console.log("click!");
</script>
```

If the handler is assigned using an HTML-attribute then the browser reads it, creates a new function from the attribute content and writes it to the DOM property.

So this way is actually the same as the previous one.

3. Events: Listeners & Handlers

These two code pieces work the same:

1. Only HTML:

```
<input type="button" onclick="alert('Click!')" value="Button">
```

Button

2. HTML + JS:

```
<input type="button" id="button" value="Button">
<script>
  button.onclick = function() {
    alert('Click!');
  };
</script>
```

Button

3. Events: Listeners & Handlers

Accessing the element: this

The value of `this` inside a handler is the element. The one which has the handler on it.

In the code below `button` shows its contents using `this.innerHTML`:

```
<input value="click me" onclick="console.log(this)" type="button">
```

3. Events: Listeners & Handlers

Possible mistakes

If you're starting to work with events – please note some subtleties.

We can set an existing function as a handler:

```
function sayThanks() {
  alert('Thanks!');
}

elem.onclick = sayThanks;
```

But be careful:
the function should be assigned as `sayThanks`, not `sayThanks()`.

If we add parentheses, then `sayThanks()` becomes is a function call. So the last line actually takes the *result* of the function execution, that is `undefined` (as the function returns nothing), and assigns it to `onclick`. That doesn't work.

```
// right
button.onclick = sayThanks;

// wrong
button.onclick = sayThanks();
```

...On the other hand, in the markup we do need the parentheses:

```
<input type="button" id="button" onclick="sayThanks()">
```

3. Events: Listeners & Handlers

addEventListener

The fundamental problem of the aforementioned ways to assign handlers – we can't assign multiple handlers to one event.

Let's say, one part of our code wants to highlight a button on click, and another one wants to show a message on the same click.

We'd like to assign two event handlers for that. But a new DOM property will overwrite the existing one:

```
input.onclick = function() { alert(1); }
// ...
input.onclick = function() { alert(2); } // replaces the previous handler
```

Developers of web standards understood that long ago and suggested an alternative way of managing handlers using special methods `addEventListener` and `removeEventListener`. They are free of such a problem.

3. Events: Listeners & Handlers

addEventListener

The syntax to remove a handler:

```
element.removeEventListener(event, handler, [options]);
```

`event`
Event name, e.g. `"click"`.

`handler`
The handler function. (must be a named function)

`options`
An additional optional object with properties:

- `once`: if `true`, then the listener is automatically removed after it triggers.
- `capture`: the phase where to handle the event, to be covered later in the chapter Bubbling and capturing. For historical reasons, `options` can also be `false/true`, that's the same as `{capture: false/true}`.
- `passive`: if `true`, then the handler will not call `preventDefault()`, we'll explain that later in Browser default actions.

3. Events: Listeners & Handlers

addEventListener

Multiple calls to `addEventListener` allow to add multiple handlers, like this:

```
<input id="elem" type="button" value="click me">

<script>
    const handler1 = () => console.log('hello from handler1');
    const handler2 = () => console.log('hello from handler2');
    elem.addEventListener("click", handler1);
    elem.addEventListener("click", handler2);
</script>
```

As we can see in the example above, we can set handlers *both* using a DOM-property and `addEventListener`. But generally we use only one of these ways.

4. Events: Event object

To properly handle an event we'd want to know more about what's happened. Not just a "click" or a "keydown", but what were the pointer coordinates? Which key was pressed? And so on.

When an event happens, the browser creates an *event object*, puts details into it and passes it as an argument to the handler.

Here's an example of getting pointer coordinates from the event object:

```
<input id="elem" type="button" value="click me">

<script>
     elem.onclick = (event) => console.log('event object: ',event)
</script>
```

4. Events: Event object

Some properties of `event` object:|

`event.type`
Event type, here it's `"click"`.

`event.currentTarget`
Element that handled the event. That's exactly the same as `this`, unless the handler is an arrow function, or its `this` is bound to something else, then we can get the element from `event.currentTarget`.

`event.clientX / event.clientY`
Window-relative coordinates of the cursor, for pointer events.

There are more properties. Many of them depend on the event type: keyboard events have one set of properties, pointer events – another one, we'll study them later when we come to different events in details.

4. Events: Event object

Object handlers: handleEvent

We can assign not just a function, but an object as an event handler using `addEventListener`. When an event occurs, its `handleEvent` method is called.

For instance:

```
<button id="elem">Click me</button>

<script>
     let obj = new Object();
     obj.handleEvent = (event) => console.log(event.type);
     elem.addEventListener('click', obj);
</script>
```

4. Events: Event object

Object handlers: handleEvent

As we can see, when `addEventListener` receives an object as the handler, it calls `obj.handleEvent(event)` in case of an event.

We could also use a class for that:

```
<button id="elem">Click me</button>

<script>
  class Menu {
    handleEvent(event) {
      switch(event.type) {
        case 'mousedown':
          elem.innerHTML = "Mouse button pressed";
          break;
        case 'mouseup':
          elem.innerHTML += "...and released.";
          break;
      }
    }
  }

  let menu = new Menu();
  elem.addEventListener('mousedown', menu);
  elem.addEventListener('mouseup', menu);
</script>
```

4. Events: Event object

Object handlers: handleEvent

The method `handleEvent` does not have to do all the job by itself. It can call other event-specific methods instead, like this:

```
<button id="elem">Click me</button>

<script>
  class Menu {
    handleEvent(event) {
      // mousedown -> onMousedown
      let method = 'on' + event.type[0].toUpperCase() + event.type.slice(1);
      this[method](event);
    }

    onMousedown() {
      elem.innerHTML = "Mouse button pressed";
    }

    onMouseup() {
      elem.innerHTML += "...and released.";
    }
  }

  let menu = new Menu();
  elem.addEventListener('mousedown', menu);
  elem.addEventListener('mouseup', menu);
</script>
```

4. Events: Event object

Summary

There are 3 ways to assign event handlers:

1. HTML attribute: `onclick="..."`.
2. DOM property: `elem.onclick = function`.
3. Methods: `elem.addEventListener(event, handler[, phase])` to add, `removeEventListener` to remove.

HTML attributes are used sparingly, because JavaScript in the middle of an HTML tag looks a little bit odd and alien. Also can't write lots of code in there.

DOM properties are ok to use, but we can't assign more than one handler of the particular event. In many cases that limitation is not pressing.

The last way is the most flexible, but it is also the longest to write. There are few events that only work with it, for instance `transitionend` and `DOMContentLoaded` (to be covered). Also `addEventListener` supports objects as event handlers. In that case the method `handleEvent` is called in case of the event.

No matter how you assign the handler – it gets an event object as the first argument. That object contains the details about what's happened.

5. Events: Bubbling & Capturing

Let’s start with an example.

This handler is assigned to `<div>`, but also runs if you click any nested tag like `<em>` or `<code>`:

```
<div onclick="alert('The handler!')">
  <em>If you click on <code>EM</code>, the handler on <code>DIV</code> runs.</em>
</div>
```

If you click on EM, the handler on DIV runs.

the handler on `<div>` run if the actual click was on `<em>`?

5. Events: Bubbling & Capturing

Bubbling

The bubbling principle is simple.

When an event happens on an element, it first runs the handlers on it, then on its parent, then all the way up on other ancestors.

It's called “bubbling”, because events “bubble” from the inner element up through parents like a bubble in the water.

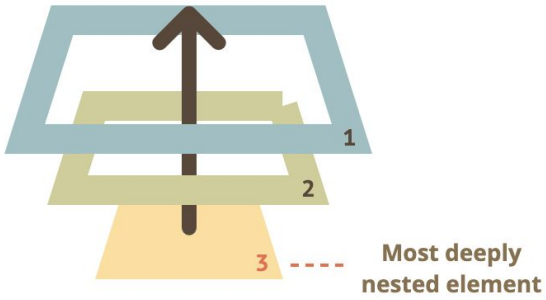
5. Events: Bubbling & Capturing

Bubbling

Let’s say we have 3 nested elements `FORM > DIV > P` with a handler on each of them:

A click on the inner `<p>` first runs `onclick`:

1. On that `<p>`.
2. Then on the outer `<div>`.
3. Then on the outer `<form>`.
4. And so on upwards till the `document` object.




```
<style>
  body * {
    margin: 10px;
    border: 1px solid blue;
  }
</style>

<form onclick="alert('form')">FORM
  <div onclick="alert('div')">DIV
    <p onclick="alert('p')">P</p>
  </div>
</form>
```

FORM

DIV

P

5. Events: Bubbling & Capturing

event.target

A handler on a parent element can always get the details about where it actually happened.

The most deeply nested element that caused the event is called a *target* element, accessible as `event.target`.

Note the differences from `this` (=`event.currentTarget`):

- `event.target` – is the “target” element that initiated the event, it doesn’t change through the bubbling process.
- `this` – is the “current” element, the one that has a currently running handler on it.

For instance, if we have a single handler `form.onclick`, then it can “catch” all clicks inside the form. No matter where the click happened, it bubbles up to `<form>` and runs the handler.

In `form.onclick` handler:

- `this` (=`event.currentTarget`) is the `<form>` element, because the handler runs on it.
- `event.target` is the actual element inside the form that was clicked.

5. Events: Bubbling & Capturing

event.target - Example *(Demo)*

It's possible that `event.target` could equal `this` – it happens when the click is made directly on the `<form>` element.

```
<html>
<head>
 <style>
   form { background-color: green; }
   div { background-color: blue; }
   p { background-color: red; }
 </style>
</head>

<body>
 Click to show <code>event.target</code> and <code>this</code> to compare:

 <form id="form">FORM
   <div>DIV
     <p>P</p>
   </div>
 </form>

 <script>
        form.onclick = function(event) { document.write('target: ',event.target.tagName, ', this: ',this.tagName, '<br>'); }
 </script>
</body>
</html>
```

5. Events: Bubbling & Capturing

Stopping bubbling

A bubbling event goes from the target element straight up. Normally it goes upwards till `<html>`, and then to `document` object, and some events even reach `window`, calling all handlers on the path.

But any handler may decide that the event has been fully processed and stop the bubbling.

The method for it is `event.stopPropagation()`.

For instance, here `body.onclick` doesn't work if you click on `<button>`:

```
<body onclick="alert(`the bubbling doesn't reach here`)">
  <button onclick="event.stopPropagation()">Click me</button>
</body>
```

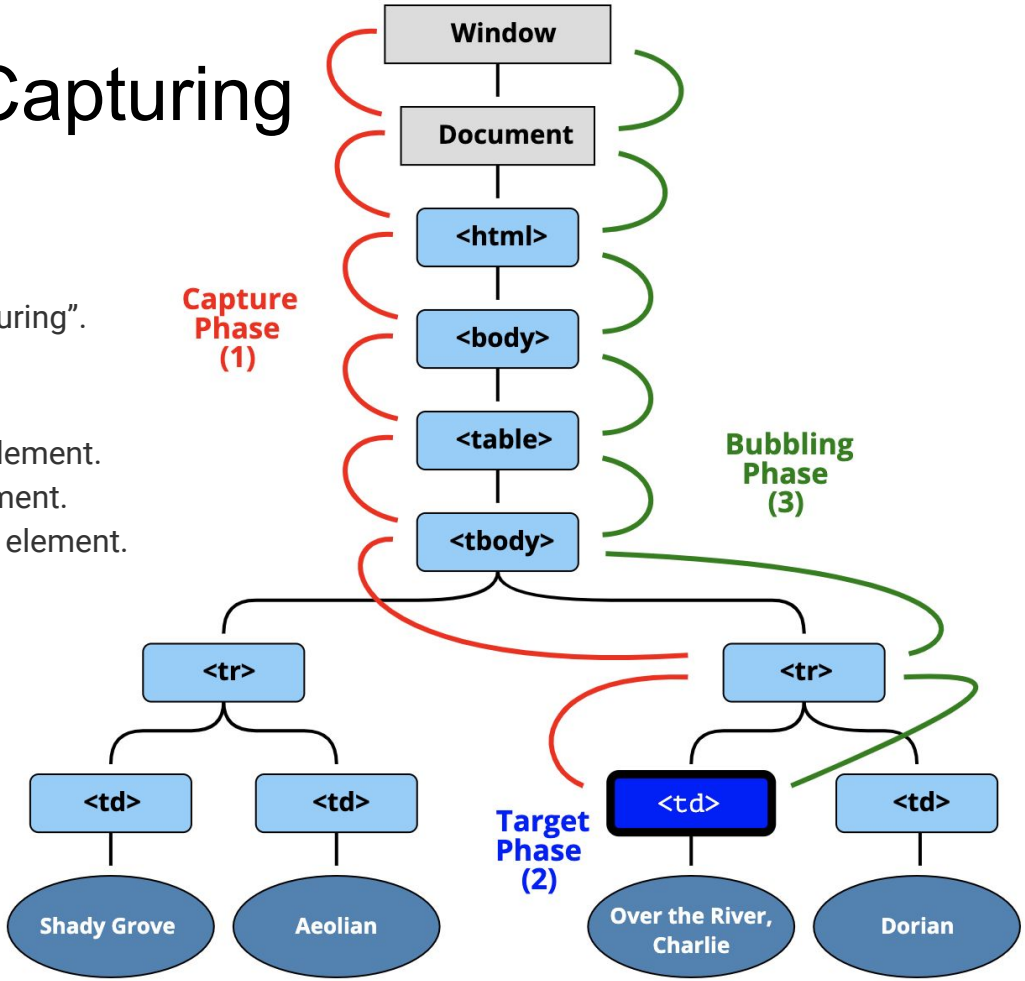
5. Events: Bubbling & Capturing

Capturing

There's another phase of event processing called "capturing".

The 3 phases of event propagation:

1. Capturing phase – the event goes down to the element.
2. Target phase – the event reached the target element.
3. Bubbling phase – the event bubbles up from the element.



5. Events: Bubbling & Capturing

Capturing

To catch an event on the capturing phase, we need to set the handler `capture` option to `true`:

```
elem.addEventListener(..., {capture: true})
// or, just "true" is an alias to {capture: true}
elem.addEventListener(..., true)
```

There are two possible values of the `capture` option:

- If it's `false` (default), then the handler is set on the bubbling phase.
- If it's `true`, then the handler is set on the capturing phase.

5. Events: Bubbling & Capturing

Capturing - (Demo)

```
<body>
<style>
 body * { border: 1px solid blue; }
</style>

<form>FORM
 <div>DIV
   <p>P</p>
 </div>
</form>

<script>
 for(let elem of document.querySelectorAll('*')) {
   elem.addEventListener("click", e => alert('Capturing:'+elem.tagName), true);
   elem.addEventListener("click", e => alert('Bubbling: '+elem.tagName));
 }
</script>
</body>
```

6. Events: Delegation

Capturing and bubbling allow us to implement one of most powerful event handling patterns called *event delegation*.

The idea is that if we have a lot of elements handled in a similar way, then instead of assigning a handler to each of them – we put a single handler on their common ancestor.

In the handler we get `event.target` to see where the event actually happened and handle it.

6. Events: Delegation

Delegation - (Demo)

NOTE:
`this.onClick` is bound to `this` in `(*)`.
That's important, otherwise `this` would reference DOM element (`elem`), not the `Menu` object, and `this[action]` would not be what we need.

Advantages:

- We don't need to write the code to assign a handler to each button. Just make a method and put it in the markup.
- The HTML structure is flexible, we can add/remove buttons at any time.

```
<body>
<div id="menu">
     <button data-action="attack">Attack</button>
     <button data-action="defend">Defend</button>
     <button data-action="flee">Flee</button>
</div>

<script>
 class Menu {
     constructor(elem) {
         this.elem = elem;
         elem.onclick = this.onClick.bind(this); // (*)
     }

     attack() { alert('attack'); }
     defend() { alert('defend'); }
     flee()   { alert('flee');   }

     onClick(event) {
         const action = event.target.dataset.action;
         if (action) {
           this[action]();
         }
     }
 }

 new Menu(menu);
</script>
</body>
```

7. Events: Custom Events

Dispatching custom events

We can not only assign handlers, but also generate events from JavaScript.

Custom events can be used to create “graphical components”. For instance, a root element of our own JS-based menu may trigger events telling what happens with the menu: `open` (menu open), `select` (an item is selected) and so on. Another code may listen for the events and observe what’s happening with the menu.

We can generate not only completely new events, that we invent for our own purposes, but also built-in ones, such as `click`, `mousedown` etc. That may be helpful for automated testing.

7. Events: Custom Events

Event constructor

Built-in event classes form a hierarchy, similar to DOM element classes. The root is the built-in Event class.

We can create `Event` objects like this:

```
let event = new Event(type[, options]);
```

Arguments:

- *type* – event type, a string like `"click"` or our own like `"my-event"`.
- *options* – the object with two optional properties:
  - `bubbles: true/false` – if `true`, then the event bubbles.
  - `cancelable: true/false` – if `true`, then the “default action” may be prevented. Later we’ll see what it means for custom events.
- By default both are false: `{bubbles: false, cancelable: false}`.

7. Events: Custom Events

dispatchEvent

After an event object is created, we should “run” it on an element using the call `elem.dispatchEvent(event)`.

Then handlers react on it as if it were a regular browser event. If the event was created with the `bubbles` flag, then it bubbles.

In the example below the `click` event is initiated in JavaScript. The handler works same way as if the button was clicked:

```
<button id="elem" onclick="alert('Click!');"> Autoclick </button>

<script>
  let event = new Event("click");
  elem.dispatchEvent(event);
</script>
```

7. Events: Custom Events

Bubbling example

We can create a bubbling event with the name `"hello"` and catch it on `document`.

All we need is to set `bubbles` to `true`:

```
<h1 id="elem">Hello from the script!</h1>

<script>
  // catch on document...
  document.addEventListener("hello", function(event) { // (1)
    alert("Hello from " + event.target.tagName); // Hello from H1
  });

  // ...dispatch on elem!
  let event = new Event("hello", {bubbles: true}); // (2)
  elem.dispatchEvent(event);
</script>
```

8. Events: Mouse - event types

We've already seen some of these events:

`mousedown/mouseup` Mouse button is clicked/released over an element.

`mouseover/mouseout` Mouse pointer comes over/out from an element.

`mousemove` Every mouse move over an element triggers that event.

`click` Triggers after `mousedown` and then `mouseup` over the same element if the left mouse button was used.

`dblclick` Triggers after two clicks on the same element within a short timeframe. Rarely used nowadays.

`contextmenu` Triggers when the right mouse button is pressed. There are other ways to open a context menu, e.g. using a special keyboard key, it triggers in that case also, so it's not exactly the mouse event.

8. Events: Mouse - events order

As you can see from the list above, a user action may trigger multiple events.

For instance, a left-button click first triggers `mousedown`, when the button is pressed, then `mouseup` and `click` when it's released.

In cases when a single action initiates multiple events, their order is fixed. That is, the handlers are called in the order `mousedown` → `mouseup` → `click`.

.

8. Events: Mouse - events order

Demo for Mouse events

```
<input onmousedown="return logMouse(event)"
       onmouseup="return logMouse(event)"
       onclick="return logMouse(event)"
       oncontextmenu="return logMouse(event)"
       ondblclick="return logMouse(event)"
       value="Click me with the right or the left mouse button" type="button">

<script>
const logMouse = function (e) {
   let evt = e.type;
   while (evt.length < 11) evt += ' ';
   console.log(evt + " button=" + e.button, 'test')
   return false;
}
</script>
```

8. Events: Mouse - button

Mouse button

Click-related events always have the `button` property, which allows to get the exact mouse button.

We usually don't use it for `click` and `contextmenu` events, because the former happens only on left-click, and the latter – only on right-click.

From the other hand, `mousedown` and `mouseup` handlers may need `event.button`, because these events trigger on any button, so `button` allows to distinguish between "right-mousedown" and "left-mousedown".

The possible values of `event.button` are:

Button state	`event.button`
Left button (primary)	0
Middle button (auxiliary)	1
Right button (secondary)	2
X1 button (back)	3
X2 button (forward)	4

8. Events: Mouse - Modifiers: shift, alt, ctrl, meta

Modifiers: shift, alt, ctrl and meta

All mouse events include the information about pressed modifier keys.

Event properties:

- `shiftKey`: `Shift`
- `altKey`: `Alt` (or `Opt` for Mac)
- `ctrlKey`: `Ctrl`
- `metaKey`: `Cmd` for Mac

They are `true` if the corresponding key was pressed during the event.

8. Events: Mouse - Modifiers: shift, alt, ctrl, meta

Demo

For instance, the button below only works on `Alt`+`Shift`+click:

```
<button id="button">Alt+Shift+Click on me!</button>

<script>
  button.onclick = function(event) {
    if (event.altKey && event.shiftKey) {
      console.log('Hooray!');
    }
  };
</script>
```

8. Events: Mouse - Coordinates: clientX/Y, pageX/Y

Coordinates: clientX/Y, pageX/Y

All mouse events provide coordinates in two flavours:

1. Window-relative: `clientX` and `clientY`.
2. Document-relative: `pageX` and `pageY`.

We already covered the difference between them in the lectures on the DOM

Quick Recap

In short, document-relative coordinates `pageX/Y` are counted from the left-upper corner of the document, and do not change when the page is scrolled, while `clientX/Y` are counted from the current window left-upper corner. When the page is scrolled, they change.

For instance, if we have a window of the size 500x500, and the mouse is in the left-upper corner, then `clientX` and `clientY` are `0`, no matter how the page is scrolled.

And if the mouse is in the center, then `clientX` and `clientY` are `250`, no matter what place in the document it is. They are similar to `position:fixed` in that aspect.

8. Events: Mouse - Coordinates: clientX/Y, pageX/Y

Demo

Move the mouse over the input field to see `clientX/clientY`:

```
document.body.onmousemove = (event) => console.log(event.clientX, event.clientY);
```

8. Events: Mouse - Prevent selection mousedown

Preventing selection on mousedown

Double mouse click has a side-effect that may be disturbing in some interfaces: it selects text.

Demo - part 1

For instance, double-clicking on the text below selects it in addition to our handler:

```
<span ondblclick="console.log('dblclick')">Double-click me</span>
```

Demo - part 2

To prevent this, the text selection is a default mouse event initialized by double click and continued by mousedown.
Thus, handle the mousedown event to prevent the default selection behavior

```
<span ondblclick="console.log('dblclick')" onmousedown="return false;">Double-click me</span>
```

8. Events: Mouse - Moving the mouse

Events mouseover/mouseout, relatedTarget

The `mouseover` event occurs when a mouse pointer comes over an element, and `mouseout` – when it leaves.

These events are special, because they have property `relatedTarget`. This property complements `target`. When a mouse leaves one element for another, one of them becomes `target`, and the other one – `relatedTarget`.

8. Events: Mouse - Moving the mouse

Events mouseover/mouseout, relatedTarget

For `mouseover`:

- `event.target` – is the element where the mouse came over.
- `event.relatedTarget` – is the element from which the mouse came (`relatedTarget` → `target`).

For `mouseout` the reverse:

- `event.target` – is the element that the mouse left.
- `event.relatedTarget` – is the new under-the-pointer element, that mouse left for (`target` → `relatedTarget`).

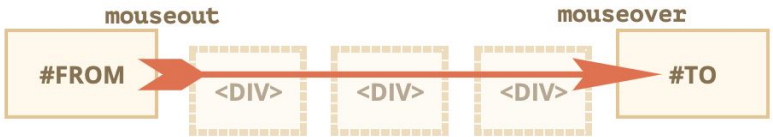
Demo

```
<div id='foo'>Foo</div>
<div id='bar'>Bar</div>
<script>
foo.onmouseover = (e) => console.log('target:',e.target, 'related:',e.relatedTarget)
bar.onmouseover = (e) => console.log('target:',e.target, 'related:',e.relatedTarget)
</script>
```

8. Events: Mouse - Moving the mouse

Skipping elements

The `mousemove` event triggers when the mouse moves. But that doesn't mean that every pixel leads to an event.
The browser checks the mouse position from time to time. And if it notices changes then triggers the events.
That means that if the visitor is moving the mouse very fast then some DOM-elements may be skipped:



If the mouse moves very fast from `#FROM` to `#TO` elements as painted above, then intermediate `<div>` elements (or some of them) may be skipped. The `mouseout` event may trigger on `#FROM` and then immediately `mouseover` on `#TO`.

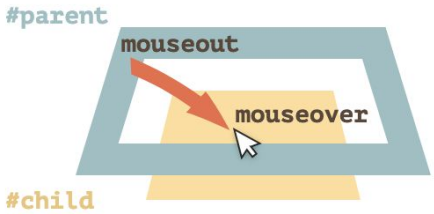
8. Events: Mouse - Moving the mouse

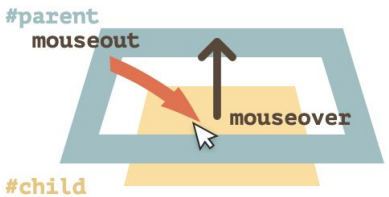
Mouseout/mouseover between parent & child

An important feature of `mouseout` – it triggers, when the pointer moves from an element to its descendant, e.g. from `#parent` to `#child` in this HTML:

```
<div id="parent">
  <div id="child">...</div>
</div>
```

If we're on `#parent` and then move the pointer deeper into `#child`, we get `mouseout` on `#parent`!





8. Events: Mouse - Moving the mouse

Mouseout/mouseover between parent & child

Demo:

```
<body id='parent'>Parent
    <div id='child'>Child</div>
</body>

<script>
parent.onmouseover = (e) => console.log(e.type ,e.target.id)
parent.onmouseout = (e) => console.log(e.type ,e.target.id)
child.onmouseover = (e) => console.log(e.type, e.target.id)
child.onmouseout = (e) => console.log(e.type ,e.target.id)
</script>
```

8. Events: Mouse - Moving the mouse

Events mouseenter and mouseleave

Events `mouseenter/mouseleave` are like `mouseover/mouseout`. They trigger when the mouse pointer enters/leaves the element.

But there are two important differences:

1. Transitions inside the element, to/from descendants, are not counted.
2. Events `mouseenter/mouseleave` do not bubble.

When the pointer enters an element – `mouseenter` triggers. The exact location of the pointer inside the element or its descendants doesn't matter.

When the pointer leaves an element – `mouseleave` triggers.

8. Events: Mouse - Moving the mouse

Events mouseenter and mouseleave

Demo:

```
<body id='parent'>Parent
    <div id='child'>Child</div>
</body>

<script>
parent.onmouseenter = (e) => console.log(e.type ,e.target.id)
parent.onmouseleave = (e) => console.log(e.type ,e.target.id)
child.onmouseenter = (e) => console.log(e.type, e.target.id)
child.onmouseleave = (e) => console.log(e.type ,e.target.id)
</script>
```

9. Events: Keyboard: types

Keyboard events

Keyboard events should be used when we want to handle keyboard actions (virtual keyboard also counts).
For instance, to react on arrow keys `Up` and `Down` or hotkeys (including combinations of keys).

Keydown and keyup

The `keydown` events happens when a key is pressed down, and then `keyup` – when it's released.

event.code and event.key

The `key` property of the event object allows to get the character, while the `code` property of the event object allows to get the "physical key code". For instance, the same key `Z` can be pressed with or without `Shift`. That gives us two different characters: lowercase `z` and uppercase `Z`. The `event.key` is exactly the character, and it will be different. But `event.code` is the same:

Key	`event.key`	`event.code`
`Z`	`z` (lowercase)	`KeyZ`
`Shift+Z`	`Z` (uppercase)	`KeyZ`

9. Events: Keyboard: types

Keydown and keyup

Demo

```
document.body.onkeyup = (e) => console.log(e.key, e.code, e.keyCode)
```

10. Events: Touch

TBD

11. Events: Document: types

DOMContentLoaded, load, beforeunload, unload

The lifecycle of an HTML page has three important events:

- `DOMContentLoaded` – the browser loaded HTML & the DOM tree is built, but external resources like pictures `<img>` and stylesheets may not yet have loaded.
- `load` – not only HTML is loaded, but also all the external resources: images, styles etc.
- `beforeunload/unload` – the user is leaving the page.

Each event may be useful:

- `DOMContentLoaded` event – DOM is ready, so the handler can lookup DOM nodes, initialize the interface.
- `load` event – external resources are loaded, so styles are applied, image sizes are known etc.
- `beforeunload` event – the user is leaving: we can check if the user saved the changes and ask them whether they really want to leave.
- `unload` – the user almost left, but we still can initiate some operations, such as sending out statistics.

Note: These events are added to the document object

11. Events: Document: types

readyState

The `document.readyState` property tells us about the current loading state.
There are 3 possible values:

- `"loading"` – the document is loading.
- `"interactive"` – the document was fully read.
- `"complete"` – the document was fully read and all resources (like images) are loaded too.

Demo

```
document.readyState
```

12. Events: Forms & Input: intro

Form properties and methods

Forms and control elements, such as `<input>` have a lot of special properties and events.

Forms are a very convenient way of handling events generated by a user. They are HTML elements defined within the DOM, & their behavior is the same for all devices.

12. Events: Forms & Input: navigation

Navigation: form and elements

Document forms are members of the special collection `document.forms`.

That's a so-called *"named collection"*: it's both named and ordered. We can use both the name or the number in the document to get the form.

```
document.forms.my; // the form with name="my"
document.forms[0]; // the first form in the document
```

When we have a form, then any element is available in the named collection `form.elements`.

Demo

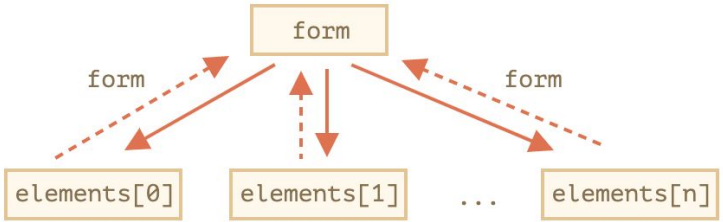
```
<form id='spam'>
    <input name='foo' value="bar">
</form>
<script>
console.log(spam.foo.value)
</script>
```

12. Events: Forms & Input: navigation

Backreference: element.form

For any element, the form is available as `element.form`. So a form references all elements, and elements reference the form.

Here's the picture:



Demo

```
<form id='spam'>
    <input name='foo' value="bar">
</form>
<script>
console.log(spam.foo.form)
</script>
```

12. Events: Forms & Input: elements

Form elements

input and textarea:
properties: `input.value` (string) or `input.checked` (boolean) for checkboxes

select and option:
properties:

1. `select.options` – the collection of `<option>` subelements,
2. `select.value` – the *value* of the currently selected `<option>`,
3. `select.selectedIndex` – the *number* of the currently selected `<option>`
4. `option.selected` - the selected value

12. Events: Forms & Input: focus/blur

Focus/Blur defined:

An element receives the focus when the user either clicks on it or uses the `Tab` key on the keyboard. There's also an `autofocus` HTML attribute that puts the focus onto an element by default when a page loads and other means of getting the focus.

Focusing on an element generally means: "prepare to accept the data here", so that's the moment when we can run the code to initialize the required functionality.

The moment of losing the focus ("blur") can be even more important. That's when a user clicks somewhere else or presses `Tab` to go to the next form field, or there are other means as well.

Losing the focus generally means: "the data has been entered", so we can run the code to check it or even to save it to the server and so on.

12. Events: Forms & Input: focus/blur

Events focus/blur

The `focus` event is called on focusing, and `blur` – when the element loses the focus.
Events `focus` and `blur` do not bubble.

Demo

In the example below:

- The `blur` handler logs the event
- The `focus` handler logs the event

```
<form id='spam'>
    <input id='foo' value="bar">
</form>

<script>
    foo.onfocus = (e) => console.log('focus',foo.value);
    foo.onblur = (e) => console.log('blur', foo.value);
</script>
```

12. Events: Forms & Input: change/input

Event: change

The `change` event triggers when the element has finished changing.

For text inputs that means that the event occurs when it loses focus.

For instance, while we are typing in the text field below – there's no event. But when we move the focus somewhere else, for instance, click on a button – there will be a `change` event:

Demo

```
<form id='spam'>
    <input id='foo' value="bar">
</form>

<script>
    foo.onchange = (e) => console.log('change',foo.value);
</script>
```

12. Events: Forms & Input: change/input

Event: input

The `input` event triggers every time after a value is modified by the user.

Unlike keyboard events, it triggers on any value change, even those that does not involve keyboard actions: pasting with a mouse or using speech recognition to dictate the text.

Demo

```
<form id='spam'>
    <input id='foo' value="bar">
</form>

<script>
    foo.oninput = (e) => console.log('input',foo.value);
</script>
```

12. Events: Forms & Input: submit

Event: submit

The `submit` event triggers when the form is submitted, it is usually used to validate the form before sending it to the server or to abort the submission and process it in JavaScript.

The method `form.submit()` allows to initiate form sending from JavaScript. We can use it to dynamically create and send our own forms to server.

Demo

```
<form onsubmit="console.log('submit!');return false">
  First: Enter in the input field <input type="text" value="text"><br>
  Second: Click "submit": <input type="submit" value="Submit">
</form>
```

12. Events: Forms & Input: submit

Method: submit

To submit a form to the server manually, we can call `form.submit()`.

Then the `submit` event is not generated. It is assumed that if the programmer calls `form.submit()`, then the script already did all related processing.

Demo:

```
let form = document.createElement('form');
form.action = 'https://google.com/search';
form.method = 'GET';

form.innerHTML = '<input name="q" value="test">';

// the form must be in the document to submit it
document.body.append(form);

form.submit();
```

13. Events: Time

13. Events: Time - scheduling

Scheduling: setTimeout and setInterval

We may decide to execute a function not right now, but at a certain time later. That's called "scheduling a call".

There are two methods for it:

- `setTimeout` allows us to run a function once after the interval of time.
- `setInterval` allows us to run a function repeatedly, starting after the interval of time, then repeating continuously at that interval.

These methods are supported in all browsers and Node.js.

13. Events: Time - scheduling

setTimeout

The syntax:

```
let timerId = setTimeout(func|code, [delay], [arg1], [arg2], ...)
```

Parameters:

- `func|code` Function or a string of code to execute.
- `delay` The delay before run, in milliseconds (1000 ms = 1 second), by default 0

Demo: *(without/with parameters to callback function)*

```
const sayHi = () => console.log('Hello');
setTimeout(sayHi, 1000)                                 //no params

const sayName = (name) => console.log(name);
setTimeout(sayName, 1000, 'foo')                        //with params
```

13. Events: Time - scheduling

Canceling with clearTimeout

A call to `setTimeout` returns a “timer identifier” `timerId` that we can use to cancel the execution.

The syntax to cancel:

```
let timerId = setTimeout(...);
clearTimeout(timerId);
```

Demo: *(without/with parameters to callback function)*

```
let timerId = setTimeout(() => console.log("never happens"), 1000);
console.log(timerId); // timer identifier

clearTimeout(timerId);
console.log(timerId); // same identifier (doesn't become null after canceling)
```

13. Events: Time - scheduling

setInterval

The syntax:

```
let timerId = setInterval(func|code, [delay], [arg1], [arg2], ...)
```

Parameters:

- `func|code` Function or a string of code to execute.
- `delay` The delay between callbacks, in milliseconds (1000 ms = 1 second), by default 0

Demo: *(without/with parameters to callback function)*

```
const sayHi = () => console.log('Hello');
setInterval(sayHi, 1000)                                  //no params

const sayName = (name) => console.log(name);
setInterval(sayName, 1000, 'foo')                         //with params
```

13. Events: Time - scheduling

Canceling with clearInterval

A call to `setTimeout` returns a “timer identifier” `timerId` that we can use to cancel the execution.

The syntax to cancel:

```
let timerId = setInterval(...);
clearInterval(timerId);
```

Demo: *(without/with parameters to callback function)*

```
let timerId = setInterval(() => console.log("never happens"), 1000);
console.log(timerId); // timer identifier

clearInterval(timerId);
console.log(timerId); // same identifier (doesn't become null after
canceling)
```

13. Events: Time - scheduling

Nested setTimeout

There are two ways of running something regularly.

One is `setInterval`. The other one is a nested `setTimeout`, like this:

```
let tick = (i=0) => {
    i+=1000
    console.log('tick', i );
    setTimeout( () => tick(i), i);
}

timerId = setTimeout(tick, 0);
```

The nested `setTimeout` is a more flexible method than `setInterval`. This way the next call may be scheduled differently, depending on the results of the current one.

13. Events: Time - scheduling

requestAnimationFrame

The syntax:

```
let requestId = requestAnimationFrame(callback)
```

Parameters:

- `callback` Function or to execute in the closest time when the browser wants to redisplay the viewport.

Demo:

```
const sayHi = () => {
    document.body.innerHTML += 'Hello<br>';
    requestAnimationFrame(sayHi)
}

requestId = requestAnimationFrame(sayHi);
```

13. Events: Time - scheduling

Canceling with cancelAnimationFrame

A call to `requestAnimationFrame` returns a “request identifier” `requestId` that we can use to cancel the execution.

The syntax to cancel:

```
let requestId = requestAnimationFrame(...);
cancelAnimationFrame(requestId);
```

Demo:

```
const sayHi = () => {
    document.body.innerHTML += 'Hello<br>';
    requestAnimationFrame(sayHi)
}

requestId = requestAnimationFrame(sayHi);
cancelAnimationFrame(requestId);
```

The End.

Coming Soon... to a Moodle near you!

Lab	JS Paint App
Homework	Browser App using DOM & Events